# $360^0$ SPACIAL MAPPING USING TIME-OF-FLIGHT

**COMPENG 2DX3 - Microprocessor System Project**

**Mark Le - 400558145**

**Submission Deadline:** Tuesday, April 7, 2026

**Instructors:** Dr. Yaser M. Haddara, Dr. Shahrukh Athar, Dr. Mohamed B. Elamien

**Statement of Originality:** As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by **Mark Le, lek37, 400558145.**

# Table of Contents

**1 Device Overview** **3**
1.1 Part A: Features . . . . . 3
1.2 Part B: General Description & Block Diagram . . . . . 5

**2 Device Characteristics Table** **6**

**3 Description** **7**

**4 Applications, System Configurations & Expected Output** **9**
4.1 Application Note . . . . . 9
4.2 Instructions . . . . . 9
4.3 Expected Output . . . . . 12

**5 Limitations** **13**
5.1 Floating Point & Trigonometric Calculations . . . . . 13
5.2 Serial Communication Rate . . . . . 13
5.3 Communication Methods . . . . . 13
5.4 Stepper Motor Speed Limit . . . . . 14

**A Circuit Schematics** **15**

**B Programming Logic Flowcharts** **16**

# List of Figures


1 Block Diagram . . . . . 5
2 VL53L1X ToF Emitter & Detector . . . . . 7
3 Code flashing instruction . . . . . 10
4 Find the port on PC . . . . . 10
5 Change the port . . . . . 11
6 Connect to MATLAB . . . . . 11
7 3D Scan of location F . . . . . 12
8 Location F picture . . . . . 12
9 Maximum serial communication rate . . . . . 13
10 Circuit schematics of the whole system . . . . . 15
11 Flowchart of the whole system . . . . . 16


# List of Tables


1 Cost of components involved . . . . . 4
2 Device Characteristics Table . . . . . 6
3 Stepper Motor Configuration Instructions . . . . . 9
4 VL53L1X Time-of-Flight sensor Configuration . . . . . 10

# 1 Device Overview

## 1.1 Part A: Features

This **LiDAR Spacial Mapper** device is a very simple interface scanning device, mainly consist of two push button that control the system, along with some relevant embedded devices that are integrated together:

- The **Texas Instrument MSP-EXP432E401Y** is a core device that communicate the distance measured for data visualization. Some noticeable features of this devices includes:
  - Operates at maximum 120 MHz bus speed. This device operates at 26 MHz.
  - 128 GPIO (General Purpose Input Output) Pins for flexible digital communications.
  - 2 on-board push buttons and 4 on-board LEDs, all GPIO controlled.
- The **VL53L1X Time-of-Flight Sensor** (ToF) acts as the main characters to acquires measurement data of the system:
  - Be able to measure up to 400 cm (4 m) of distance, at maximum sampling frequency of 30 Hz.
  - Operating voltage ($V_{IN}$) of 2.6 V to 5.5 V.
  - 16-bit distance reading in millimeters using $I^2C$ communication protocol.
- The **28BYJ-48 Stepper Motor** has the following features:
  - Operating voltage of 5V DC
  - 4 phases, with gear ratio of 64:1
  - 4 LEDs on the driver for indicating current phase.
- Two off-board push buttons has the following features
  - Button 1: Start/Stop button. This button is responsible for start and stop the motor spin.
  - Button 2: Home button, for returning to the original position of scan when pressed.
  - Additionally, there is also a reset button on the microcontroller **Texas Instrument MSP-EXP432E401Y** to activate this device.
- System Communication protocol includes:
  - Time-of-Flights (ToF) sensor utilizes $I^2C$ protocol to send the measurement data captured to the microcontroller **Texas Instrument MSP-EXP432E401Y**.

  - **UART** protocol for transmitting the data from the microcontroller, to **MATLAB** (receiver) for serial data reading, at baud rate of 115200 bps.

- Cost of development (all in Canadian Dollars - CAD)

Table 1: Cost of components involved

| **Component** | **Quantity** | **Unit price** | **Total** |
|---|---|---|---|
| TI MSP-EXP432E401Y Microcontroller | 1 | $171.27 | $171.27 |
| Jumper Wires | 1 (pack) | $2.84 | $2.84 |
| 28BYJ-48 Stepper Motor + ULN 2003 Driver Board | 1 | $5.82 | $5.82 |
| VL53L1X Time-of-Flight sensor | 1 | $12.71 | $12.71 |
| Push buttons | 2 | $0.37 | $0.74 |
| 1kΩ resistors | 2 | $0.16 | $0.32 |
| **Total** | | | **$193.70** |

## 1.2 Part B: General Description & Block Diagram

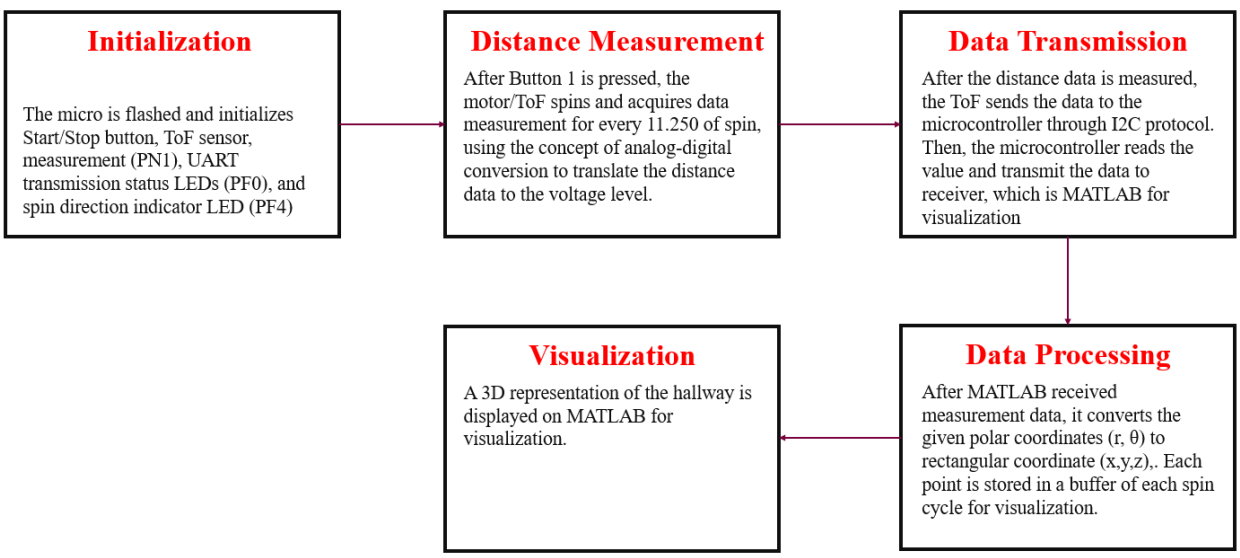
When the microcontroller is set/reset, it will wait for pressing Button 1 (start/stop). If Button 1 is pressed, the motor starts to spin and the ToF starts to acquires the measurement data. The measurement is captured using the concept of analog-to-digital conversion, for every $11.25^0$ of rotation, in which 32 measurements are made in each spin rotation.

Once the measurement data is captured, the $I^2C$ protocol is used between the microcontroller and the ToF module. The ToF acts as a $I^2C$ slave to send data to the microcontroller, a $I^2C$ host. Then, UART protocol is utilized to transmit the measurement data from the microcontroller (Tx) to MATLAB (Rx) for data serial reading, at a baud rate of 115200 bps.

Once MATLAB receives the measurement data, it will parse the data transmitted, including angle $\theta$, and distance $d$. Then, by applying simple trigonometry, polar coordinates are transformed into rectangular coordinates for visualization purpose. In particular, the width $y$ is computed as $y = d\cos\theta$, and height $z$ is computed as $z = d\sin\theta$.

Block diagram:

Figure 1: Block Diagram



# 2 Device Characteristics Table

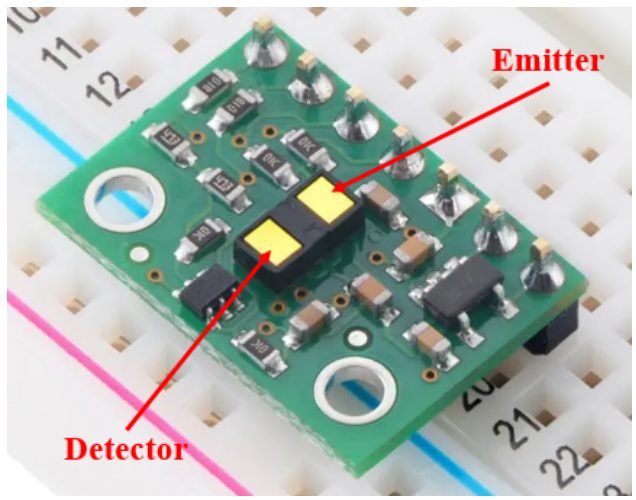
Table 2: Device Characteristics Table

| **Feature** | **Description** |
|---|---|
| **General** | |
| UART Transmission Settings | 115200 bps 8N1 |
| I2C Transmission Speed | 100000 bps |
| Datapoints per scan cycle | 32 |
| Angle between measurements & UART transmission | $11.25^0$ |
| Serial port | COM3 |
| **Microcontroller TI MSP432E401Y** | |
| Clock speed | 26 MHz Bus speed |
| Measurement status LED | PN1 (D1) |
| UART transmission status LED | PF0 (D4) |
| Motor/ToF spin direction status LED | PF4 (D3) |
| Push Button 1 - Start/Stop | PL0 |
| Push Button 2 - Homing | PM0 |
| Bus speed tester | PK0 |
| **Stepper Driver Board ULN 2003** | |
| In1 | PH0 |
| In2 | PH1 |
| In3 | PH2 |
| In4 | PH3 |
| V+ | 5V |
| V- | GND |
| **VL53L1X Time-of-Flight Sensor** | |
| $V_{IN}$ | 3.3V |
| GND | GND |
| SDA | PB3 |
| SCL | PB2 |

# 3 Description

The VL53L1X Time-of-Flight sensor operates by emitting a light beam from the emitter. This beam of light then hit the target object and bounce back to the detector. Figure 2 shows the emitter and the detector of the ToF sensor.

Figure 2: VL53L1X ToF Emitter & Detector




Let $t$ be the total time delay between the emission and reception, then the distance $d$ is computed as $d = c(t/2)$, where $c$ is the speed of light ($c = 299792458$ m/s). Once the distance is acquired, the sensor acts as a transducer to convert the measurement into electric signal, then the signal is conditioned for analog-to-digital conversion (ADC). The digital information converted is then transmitted to the microcontroller through I2C communication protocol. LED D1 will flash to indicate distance measurement status. Then, the microcontroller acts as a UART transmitter (Tx) to transmit the measurement data to the UART receiver (Rx), which is MATLAB, for data processing. LED D4 will flash to indicate UART transmission status. The ToF sensor acquires measurement data every $11.25^0$ of rotation, or 32 measurements per full rotation.

In order to activate the ToF, the microcontroller must invoke its boot function. Initially, the system will not start scanning because the microcontroller waits for button 1 (PL0) to be pressed, through polling method. Once Button 1 is pressed, the C code control the distance measurement. It will check whether the motor spin for $11.25^0$ or not, by taking the current number of steps, out of 2048 (`stepCount`) with the modulo of 64. If `stepCount % 64 = 0`, then the microcontroller will wait for the ToF booted, tells it that the measurement is ready to take. Then, the measurement data is acquired, including angles (in degree), range status (tells how accurate the data), distance (in mm), ambient rate, spad number, all comes from the ToFs API function. Depth and distance are the two variables that are controlled within the C program. Distance is acquired through VL531X API function, and depth are incremented by 700 mm, since 10 cycles of rotations is assumed across the hallway of 7m (7000 mm).

Once one full rotation is made, to avoid mechanical problems, the motor/ToF will spin backwardly to ”untwist” the wires. LED D3 (PF4) will be turned on to indicate backward spin.

Once the microcontroller transmit the measurement data to MATLAB through UART, MATLAB is now responsible for data processing. First, MATLAB connects to the COM3 serial port and open it for serial communications. The measurement data is configured to have the format of [angle, RangeStatus, Distance, Depth, AmbientRate, SpadNum]. For MATLAB data processing, only angle, Distance, and Depth are used. Therefore, this vector is parsed to obtain those three parameters. There are no further processing to the angle, since the C code already handle floating point numbers. A debugging message is also used to track the data reading from the microcontroller to MATLAB via UART.

```
% print message showing received values, for debugging purpose!
fprintf('Received values - Theta: %.2f, Range Status: %d, R: %.2f, Depth: %.2f\n',
theta, rangeStatus, r, depth);
```

Then, the polar coordinates $(r, \theta)$ given by the raw vector is transformed to Cartesian coordinates, using **Math 1ZB3 - Engineering Mathematics II-A** course knowledge. The width $y$ is calculated as $y = r\cos\theta$, and the height $z$ is computed as $z = r\sin\theta$. The depth $x$ increments by 700 mm after every spin cycle. Finally, the width and height will be plotted along the $yz$-plane.

For example, if the motor is at $78.75^0$ and the distance acquired by the ToF is found out to be 1450 mm, at the 2nd cycle of spin (i.e first counterclockwise spin, which is at the depth of 700 mm), then $y = r\cos\theta = 1450\cos(78.75^0) \approx 283$ mm, $z = r\sin\theta = 1450\sin(78.75^0) \approx 1422$ mm. Therefore, this data point will correspond to (700, 283, 1422) coordinate on the plot.

A **layer** is outputted on MATLAB plot after each cycle, which includes 32 data points connected together, at a particular depth level. Once the motor change its direction of spin, depth level ($x$-coordinate) is incremented by 700 mm (handled by C code), and the next layer is outputted once the next cycle is done. The next layer will be connected to the previous layer to give better visualization of the plot.

# 4 Applications, System Configurations & Expected Output

## 4.1 Application Note

The LiDAR Spatial Mapping System can be applied in real-world environment scanning. By acquiring measurement data at $360^0$, a basic 3D map of the space is produced. Such application can support autonomous robots or mobility devices that need to navigate safely indoors where GPS is not always available.

Another relevant application is assistive technology devices. The system is being able to detect nearby surfaces or obstacles by knowing the horizontal and vertical coordinates of the obstacles. Further integrate this project as a object detection system will aid people with disability to accommodate them in daily indoor navigation.

One common another example would be safety distance detection among cars. All modern cars model has their safety distance feature. The feature works by detecting all objects surround it, by computing all spacial coordinates in order to make appropriate decision such as alerting the drivers.

## 4.2 Instructions

**Hardware Configuration**: to avoid any confusion, the whole hardware system configuration is placed under **Appendix A: Circuit Schematics**. Please mindful of the following:

1. Button 1 and 2 configuration: configure the circuitry of button 1 and 2 by following the circuit schematics shown as Figure 10 - Circuit Schematics of the whole system, under **Appendix A: Circuit Schematics**. Button 1 is connected to GPIO Port PL0, and button 2 is connected to GPIO Port PM0, on the microcontroller, using a **pull-up** network with 1kΩ resistors.

2. Stepper motor configuration: refer to the table below. It can also be referred to Table 1: Device Characteristics Table as well.

Table 3: Stepper Motor Configuration Instructions

| **Stepper Driver Board ULN 2003** | |
|---|---|
| In1 | PH0 |
| In2 | PH1 |
| In3 | PH2 |
| In4 | PH3 |
| V+ | 5V |
| V- | GND |

3. VL53L1X ToF configuration: refer to this table below for configuration instruction:

Table 4: VL53L1X Time-of-Flight sensor Configuration

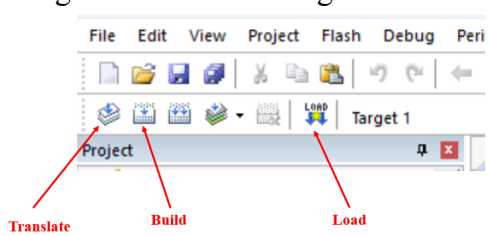
| **VL53L1X Time-of-Flight Sensor** | |
|---|---|
| $V_{IN}$ | 3.3V |
| GND | GND |
| SDA | PB3 |
| SCL | PB2 |

4. Connect the microcontroller to your computer.

**Software configuration**: it is assumed that Keil $\mu$Vision IDE, MATLAB is downloaded. Ensure that the microcontroller is connecting to your PC.

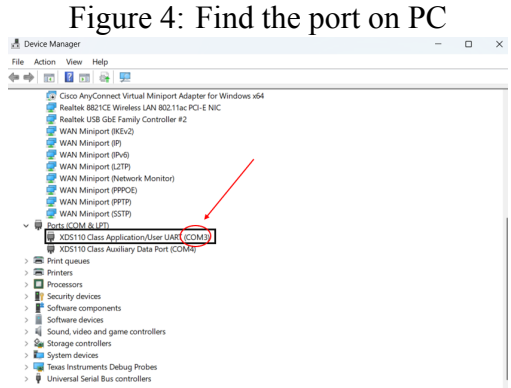
1. Open Keil $\mu$Vision, open the project `2dx_studio_8`. Once the IDE is opened, double click on `2dx_studio_8.c` file. Click `Translate` $\rightarrow$ `Build` $\rightarrow$ `Load`. Refer to the figure below to know which button is which.

Figure 3: Code flashing instruction



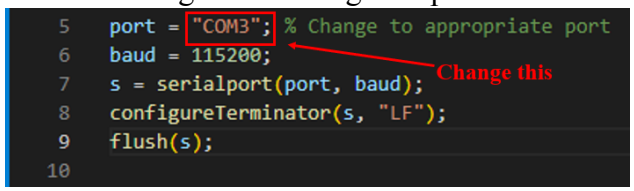

2. Go to search bar, search for **Device Manager** and hit enter. Look under **Ports (COM & LPT)** and note down the COM # next to **XDS110 Class Application/User UART**. The example below shows COM3, but each computer have different number. Refer to the figure below:

Figure 4: Find the port on PC



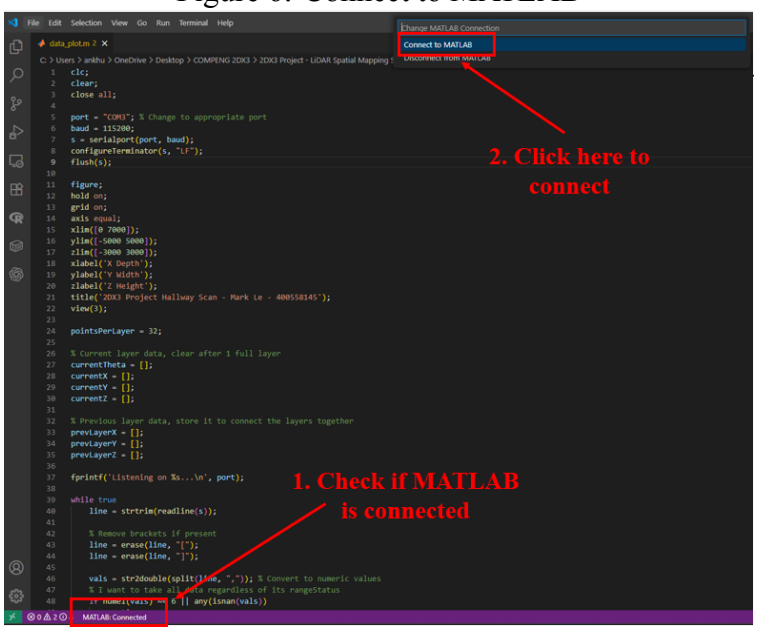
3. Open the MATLAB file `data_plot.m`, go to line 5 of the code and change to your own corresponding port. Refer to the figure below:

Figure 5: Change the port




4. **If you are using MATLAB on VSCode**, please make sure that MATLAB is connected. Refer to figure below:

Figure 6: Connect to MATLAB




5. Reset the microcontroller and run the MATLAB code. The system is ready to use. After MATLAB generate one layer, step forward 70 cm (approximately) to obtain the next cycle of scan.

## 4.3 Expected Output

The expected finalized output of the device can be found by comparing the scanned location F to the 3D representation shown below. The system assume each cycle of scan are 70 cm apart (i.e depth is incremented by 700 mm after every spin cycle done), since the hallway is 7m long.

Figure – was able to capture the scan data relatively well. The hallway starts at a width, and then it got extended a bit when it met the elevator, and finally reaches the junction, where the sensor wasn't able to capture distance that is longer than 4m. This is not a technical problem, rather a limitation of the measure ability of the ToF sensor. Figure – shows what does the hallway looks like. The $x$-axis is defined as depth level (i.e as the position of the ToF is moved toward the end of the hallway). The $y$-axis is defined as the width (point toward the wall on left and right). The $z$-axis is defined as the height (point toward the ceiling and floor).

Figure 7: 3D Scan of location F

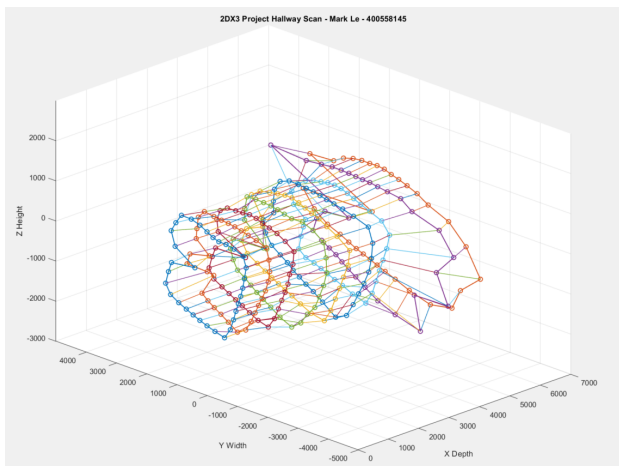



Figure 8: Location F picture

# 5 Limitations

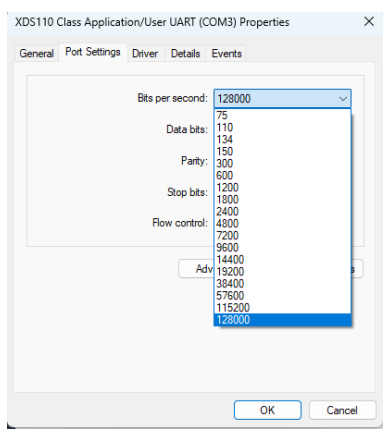
## 5.1 Floating Point & Trigonometric Calculations

The TI MSP-EXP432E401Y microcontroller utilizes a built-in Floating Point Unit (FPU) that supports 32-bit single-precision calculation such as addition, substraction, multiplication, and division. It also support conversion between different data types, like integer and floats. Additionally, for coordinate transformation purpose, trigonometric function like sin and cos are also used to compute the coordinates. These computations can be done fairly quickly with minimal delays and high precision. To ensure higher precision, sin and cos function are computed using MATLAB, which provide double precision (64-bits) for better data precision and visualization.

## 5.2 Serial Communication Rate

The maximum serial communication speed that my PC can support is 128 000 bps. Since the **XDS110 Class Application/User UART** port (COM3) is used for serial communication, it is easy to find out the maximum communication speed by going to `Properties` → `Port Settings` → `Bits per second` → select the maximum value. Refer to the figure below:

Figure 9: Maximum serial communication rate




## 5.3 Communication Methods

The communication protocol between the microcontroller and the ToF module is I2C. The measurement data is send from the ToF sensor as a I2C slave, then it is transmitted to the microcontroller as I2C host through SDA (data) line. The communication is synchronized by the SCL (clock) line. The communication rate is 100 000 bps.

## 5.4 Stepper Motor Speed Limit

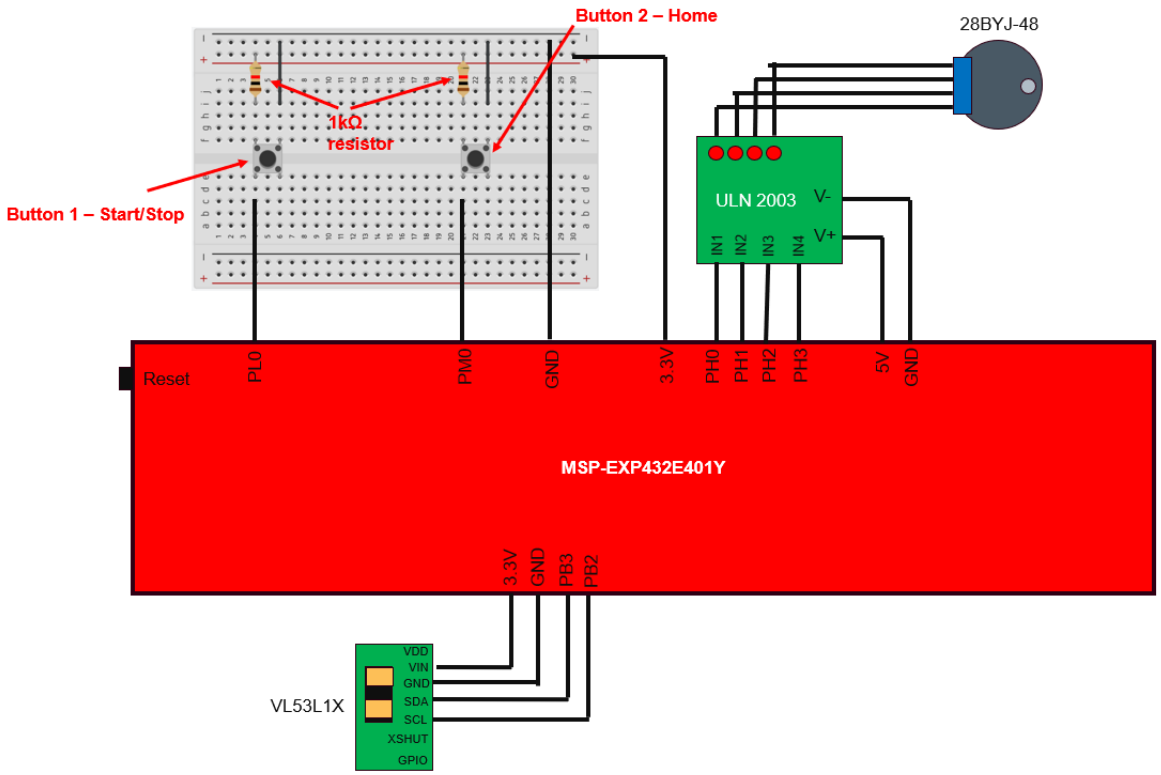
Upon developing the project, the minimum time delay between each phases of the stepper motor is found out to be 1.5 ms. This was done by decreasing the argument in the delay function `void SysTick_Delay10us(delay)`. This function prototype is not provided in the file `SysTick.c`, so I had to manually wrote it.

```
void SysTick_Wait10us(uint32_t delay){
  uint32_t i;
  for(i=0; i<delay; i++){
    SysTick_Wait(260);  // wait 10us (26 MHz clock)
  }
}
```

To test the speed, first the argument is set to be 1000, then decrease logarithmically, from 500, 250, 125. 125 is when the motor stop functioning, so it was increased until the motor could spin normally with no sign of malfunctioning, which is 150. By setting the argument to 150, the minimum delay between two phases of the motor is 1500 $\mu$s = 1.5 ms.

# A Circuit Schematics

Figure 10: Circuit schematics of the whole system




**Note:**

- Some of the GPIO ports are in fact, not in the corresponding position from the above figure. This is for visualization purpose only.
- Button 1 and Button 2 utilize a pull-up network (PUN) principle.

# B Programming Logic Flowcharts

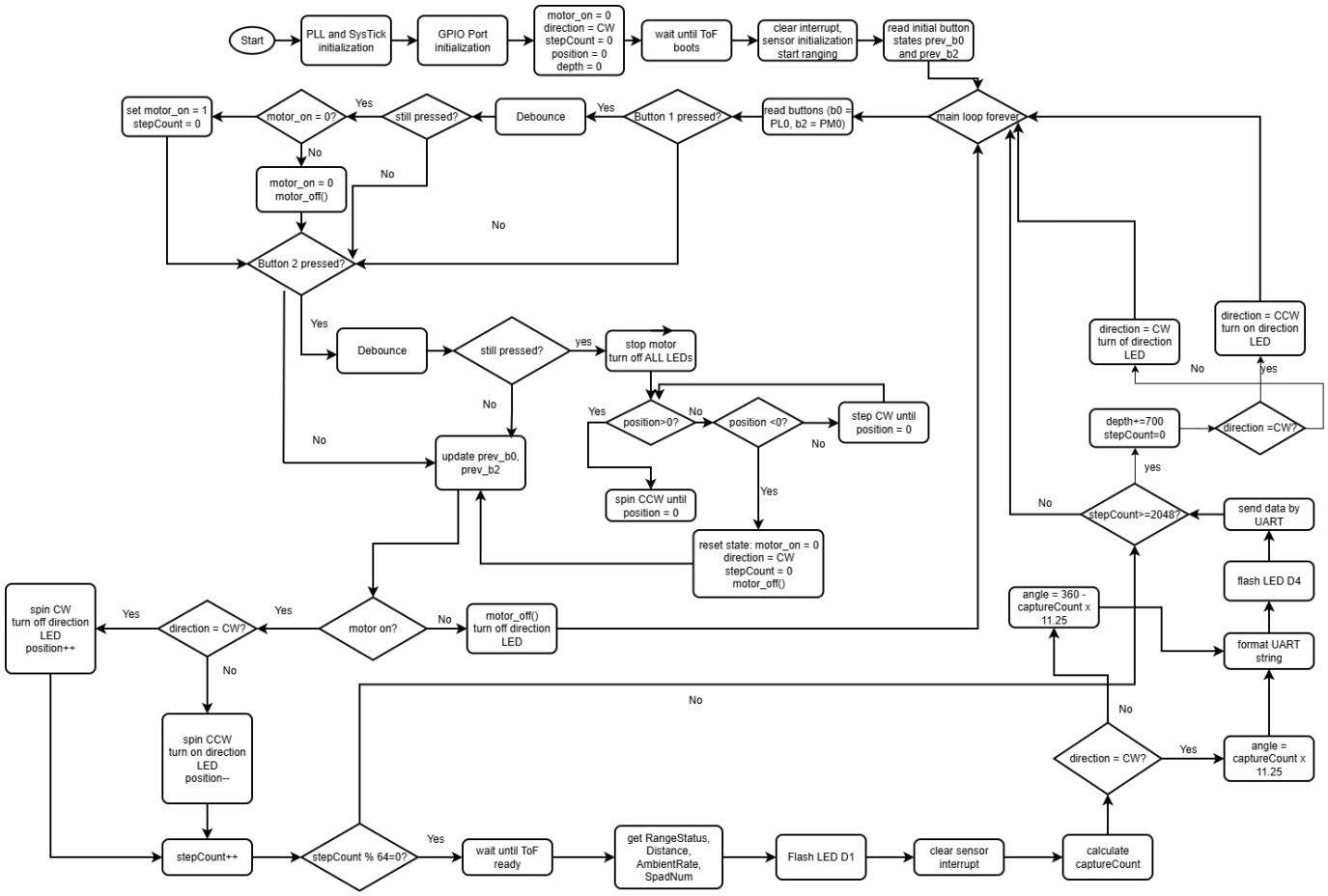



Figure 11: Flowchart of the whole system